

ASTRE - Lab Assignment 4

Amy van der Meijden (5301513) and Jimmy Oei (6540031)

June 2026

Task 1

What is wrong with an inconsistent observation table? Can you explain it in terms of the mealy machine that can be created from the observation table?

An inconsistent observation table means that two prefixes have the same row in the observation table, but there is some extension that gives different outputs. This means the suffixes that are used in the observation table are insufficient to distinguish the states of the two prefixes. This means that they are expected to give the same state, but since they are different states, it would result in a non-deterministic model: the same state would produce different outputs for the same input.

What is wrong with an observation table that is not closed? Can you explain it in terms of the mealy machine that can be created from the observation table?

An observation table that is not closed contains a row in $S \cdot A$ (prefix S extended with A) that does not match any equivalent row in S (prefix only). In terms of the mealy machine, this means that the machine contains states with undefined behavior as a transition would need to point to a state that does not exist.

Task 2

When the equivalence checker does not return a counterexample, you are done learning, but is your model guaranteed to be correct? Why (not)?

When the equivalence checker does not return a counterexample, the model is not guaranteed to be correct, because only a finite number of traces are checked. It is possible that a counterexample still exists, but was simply not encountered during the exploration.

Task 4

When the w-method equivalence checker does not return a counterexample, you are done learning, but is your model guaranteed to be equivalent to the system under learn? Why (not)?

No, the model is not guaranteed to be equivalent. The w-method is a finite equivalence test and only checks membership queries for a specific set of input sequences. If the real system has more states than the bound assumed by w , or if w is too small, then the method may miss a difference.

Task 5

Plot the number of states over time for each RERS problem and the ProblemPin. Include the final renders of the problems in your report.

Below, in Figure 1, one can find a plot of the number of states over the hypothesis iterations. Figure 2 to 6 show the final renders of the problems.

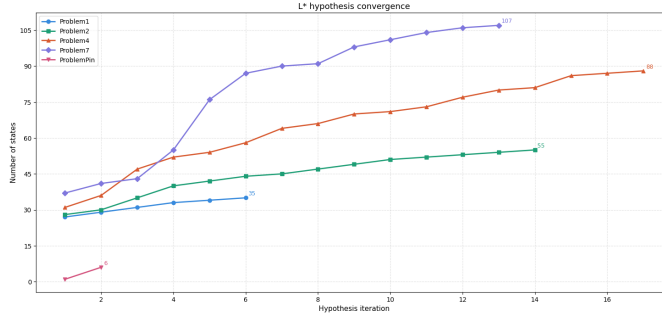


Figure 1: States over time

The ProblemPin represents a keypad where someone must enter a passcode. View the mealy machine of this problem and explain if the model matches your expectations. Do you think this is 'secure'? Why/why not?

No, this does not seem secure. First of all, the figure reveals the PIN length and structure. Moreover, there is no penalty in the system repeated failed attempts, allowing an adversary to brute-force the PIN.

Task 6

Is your method able to find the same models as TTT? Why, or why not?

Yes, our implementation of L^* finds the same models as the TTT algorithm. Both algorithms converge to identical state counts across all problems: 35 states for Problem 1, 55 for Problem 2, 87 for Problem 4, 107 for Problem 7, and 6 for ProblemPin. They are the same because both algorithms should converge to the minimal mealy machine of the system if both algorithms are correct, which they should be.

Compare the runtime, as well as the number of membership queries of your method to the state-of-the-art. How does your implementation compare to the state-of-the-art?

Table 1 shows the runtime and membership query counts for our L^* implementation and the state-of-the-art TTT algorithm across all problems. Our L^* implementation outperforms TTT in both runtime and membership query count across all

Problem	Algorithm	Queries	Time (ms)	States
Problem 1	L^*	889,405	8,814	35
Problem 1	TTT	4,691,435	44,873	35
Problem 2	L^*	3,556,857	36,112	55
Problem 2	TTT	12,612,199	138,987	55
Problem 4	L^*	3,609,452	36,630	87
Problem 4	TTT	22,985,351	273,075	87
Problem 7	L^*	22,321,088	231,006	107
Problem 7	TTT	170,880,976	1,867,046	107
ProblemPin	L^*	894,291	9,201	6
ProblemPin	TTT	2,760,938	36,511	6

Table 1: Comparison of L^* and TTT across all problems ($w=3$, $w=4$ for ProblemPin).

problems. TTT is designed to minimise the number of membership queries used to build the hypothesis, at the cost of requiring more rounds before converging. This trade-off pays off when individual membership queries are expensive. However, in this experiment the equivalence check dominates the total cost, since it is expensive and is executed every round. Since L^* requires fewer rounds, it calls the W-method less often and therefore outperforms TTT.

References

- [1] Source code repository: <https://github.com/AmyAmyy/ASTRE26/tree/Lab4>

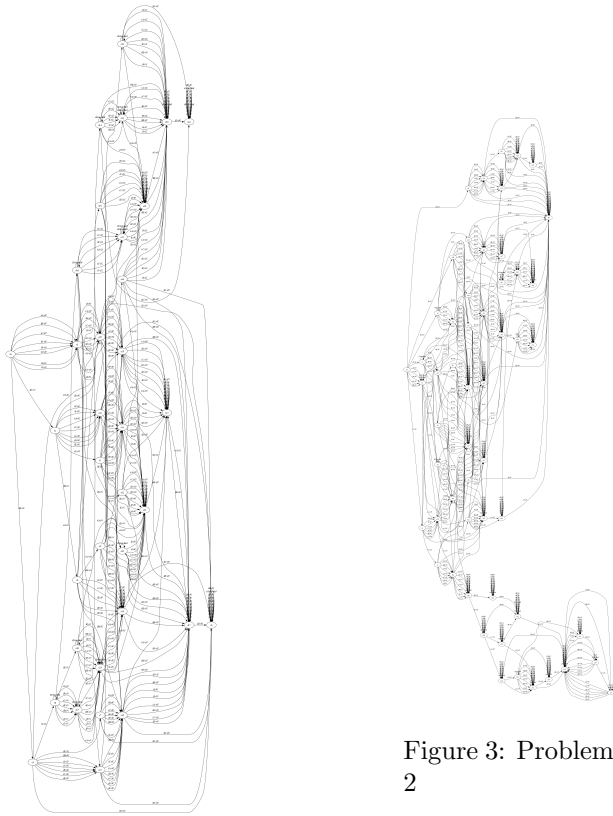


Figure 2: Problem
1

Figure 3: Problem
2

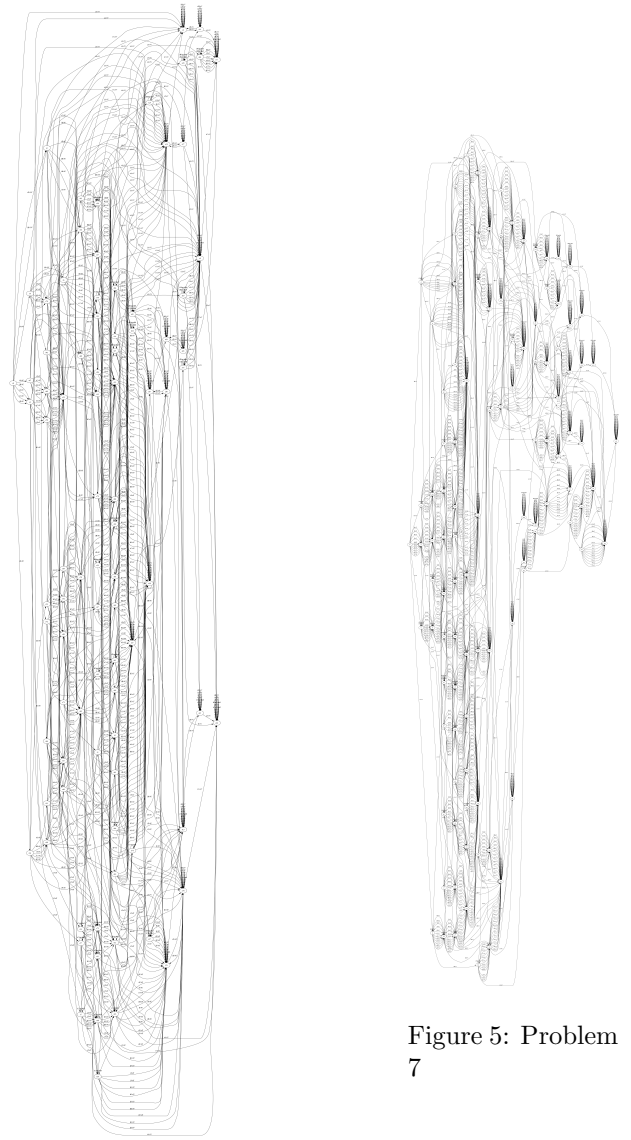


Figure 4: Problem
4

Figure 5: Problem
7

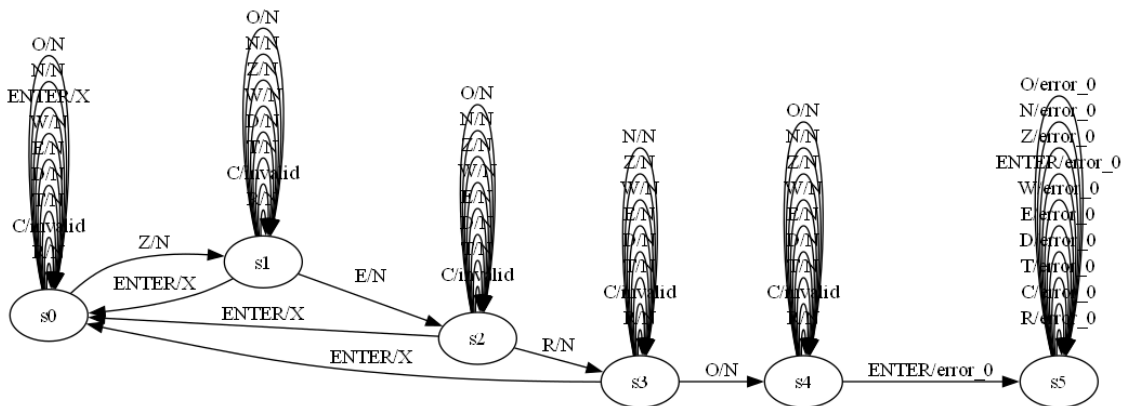


Figure 6: ProblemPin